

Reliability & Fallback Design Patterns for Multi-Agent LLM Systems

Anonymous Author(s)

Anonymous Institution

Abstract

When a multi-agent LLM pipeline fails partway through a run, most systems either stop or blindly retry the same stage. Neither is a real reliability strategy. We compare six conditions on a fixed Planner-Retriever-Synthesizer-Formatter pipeline: no fallback, blind retry, and four targeted recovery patterns (tool-grounded retry, deterministic checkpointing, graceful degradation, and cross-agent verification). Across 40 HotpotQA multi-hop questions and multiple trials, three targeted patterns significantly beat the no-fallback baseline on pass rate ($p < 0.01$), at modest added cost. Blind retry and cross-agent verification do not significantly beat doing nothing, despite costing more and stretching tail latency by 3x. The three winning patterns are statistically indistinguishable from each other, suggesting several different designs converge on similar reliability gains rather than one clear winner. We release code, data, and logs for reproduction.

1. Introduction

Multi-agent LLM pipelines fail in the middle of a run as often as they fail at the end. A Planner invents bad sub-questions. A Retriever returns nothing useful. A Synthesizer emits broken JSON or cites a document that was never retrieved. When that happens, most systems do one of two things: stop, or blindly re-run the same stage with the same prompt and hope the next sample is better.

Neither option is a reliability strategy. Stopping wastes the work already done. Blind retry burns tokens and wall time on correlated errors. Production teams are shipping multi-agent stacks anyway--support bots, research agents, tool-using workflows--and the recovery logic is usually ad hoc: a retry counter here, a second "checker" agent there, with little evidence for which choice actually cuts end-to-end failure, or what it costs in latency and dollars.

This paper asks a direct question: **which fallback strategies most reduce total task failure in multi-agent LLM pipelines, and what latency and cost tradeoff does each carry?**

We answer it on one fixed testbed: a Planner -> Retriever -> Synthesizer -> Formatter multi-hop QA pipeline on LangGraph with Claude Sonnet. We compare six conditions on the same 40 HotpotQA questions--no fallback (B0), blind retry (B1), tool-grounded retry (P1), deterministic checkpointing (P2), graceful degradation (P3), and a cross-agent verification gate (P4)--and measure them together on pass rate, p50/p95 latency, and cost per question.

The headline is not subtle. Targeted fallbacks that use external checks or trusted checkpoints (P1, P2, P3) significantly beat doing nothing. Blind retry and the fancier verifier gate do not, despite costing more and stretching the latency tail. P1, P2, and P3 also land statistically indistinguishable from each other: several different designs buy similar gains. The expensive second-agent gate is not automatically worth it.

The rest of the paper is organized as follows. Section 2 situates the patterns against retry, verification, production RCA, and agent-failure surveys. Sections 3-4 describe the pipeline, conditions, dataset, and trial design. Section 5 reports the pooled comparisons and tradeoff figures. Sections 6-7 say what that means for builders and where the claims stop.

2. Related Work

2.1 Retry, fallback, and self-correction

A lot of reliability work for LLMs is some form of try again. Wang et al. (Self-Consistency, 2023) sample multiple reasoning paths and majority-vote the answer. That cuts failure on reasoning tasks, but cost and latency scale with the number of samples. Reflexion (Shinn et al., 2023) stores natural-language reflections from environment feedback and retries with that memory. It helps when you have external scalar or binary feedback. It does not compare several fallback policies on cost and latency together.

CRITIC (Gou et al., 2024) is closer to what we use in the tool-grounded condition. The model verifies and revises using external tools (search, code execution), in a verify-then-correct loop. Gains depend on the quality of those tool

signals, and each iteration adds calls and latency.

Two negative results matter for our design. Huang et al. (2024) show that *intrinsic* self-correction--asking the model to fix itself with no external feedback--often fails to help and can make reasoning worse. Earlier positive numbers often leaned on oracle labels or weak baselines. Kamoi et al. (2024) survey the self-correction literature and reach the same bottleneck: feedback quality. Reliable correction needs external signals, task-specific checks, or fine-tuning--not prompted self-evaluation alone. That is why our targeted patterns (P1-P3) feed stages checkable facts (retrieval miss, bad citation ID, broken schema) rather than "please fix your answer."

FrugalGPT (Chen et al., 2024) is the main prior work that treats cost and quality as a joint problem. It cascades to stronger models only when a scoring function says the current answer is unreliable. That is model routing for single queries. It is not a comparison of orchestration-level fallbacks when a multi-agent pipeline fails mid-graph, and it does not report p95 latency under agent handoffs.

2.2 Cross-agent and self-verification

Multiagent Debate (Du et al., 2024) has several LLM instances propose answers, critique each other, and converge. Factuality and reasoning improve over a single agent. Token cost and round-trips grow with the number of agents and rounds. That is the same tradeoff space as our cross-agent verification condition (P4), but debate is usually the whole method, not a gate at one handoff inside a fixed pipeline.

LLM-as-a-Judge work (Zheng et al., 2023) shows strong models can score outputs with high correlation to human preference, and also that judges carry biases (position, verbosity, self-enhancement). Any verifier-gated retry inherits those issues. If the judge is soft or biased, you get false accepts or wasted rejects. We treat that as a reason to measure P4 on failure, latency, and cost together--not only on whether the gate "looks careful."

2.3 Production reliability engineering

Production work is mostly about finding what went wrong after deploy, not about choosing an in-pipeline fallback policy. LLMRCA (Tan et al., 2026) combines metrics, logs, and traces to diagnose performance regressions and silent quality faults in LLM/RAG apps. That is useful measurement substrate: many failures return HTTP 200 and still wrong. Roy et al. (2024) study ReAct-style agents for cloud incident root-cause analysis, using retrieval and the same diagnostic tools on-call engineers use. Both lines show agentic recovery in operations. Neither benchmarks a menu of orchestration fallbacks (retry vs checkpoint vs degrade vs verify) on task failure, p95 latency, and dollar cost in one controlled multi-agent testbed.

2.4 Failure characterization surveys

Wang et al. (agent survey, 2024) survey LLM-based autonomous agents and organize construction around profile, memory, planning, and action. That map is useful for where a fallback can hook in (re-plan, re-act, refresh memory). It does not empirically compare mitigation strategies.

Song et al. (2026) survey LLM reasoning failures by reasoning type and failure class, with mitigation ideas per class. That helps say *what* a fallback should catch. It stops short of ranking orchestration policies under a shared cost and latency budget.

Zhu et al. (2025) (AgentDebug) attribute agent errors to modules (memory, reflection, planning, action, system) and show that isolating a root cause and re-rolling from there beats undifferentiated retry on their benchmark. That is the closest prior on recovery *inside* agent pipelines. It validates one targeted recovery idea. It does not put several fallback families on the same axes of failure reduction, latency, and cost.

2.5 Where this leaves a gap

Taken together, the literature already offers the pieces: vote and retry (Self-Consistency), reflect-and-retry (Reflexion), tool-grounded critique (CRITIC), model cascades (FrugalGPT), multi-agent debate, and LLM-as-judge gating. Most papers validate one technique on accuracy or task success. Few report end-to-end failure rate, p95 latency, and dollar cost on the same runs. Huang and Kamoi show why blind intrinsic self-fix is a bad default. CRITIC, Reflexion, and debate show recovery can work when feedback is well-sourced. What is still missing is a side-by-side answer to *which fallback to use when*, under a fixed multi-agent workload and a joint reliability budget.

Production RCA work diagnoses deployed systems after the fact. Agent and reasoning-failure surveys explain failure modes and where hooks might attach. FrugalGPT prices model routing, not cascading errors and verification gaps inside a Planner->Retriever->Synthesizer->Formatter graph. AgentDebug shows targeted recovery can beat blind retry for one mechanism.

That is where this paper comes in. We treat fallback strategies as a design space--no fallback, blind retry, tool-grounded retry, deterministic checkpointing, graceful degradation, and cross-agent verification--applied at orchestration boundaries in one multi-agent QA pipeline, and we measure them together on pass rate, latency, and cost. Prior work proposes and validates individual patterns. We compare those options side by side and show what each costs.

3. Method

3.1 Testbed

We evaluate fallback patterns on a fixed multi-agent multi-hop QA pipeline. The pipeline has four stages, in order: **Planner**, **Retriever**, **Synthesizer**, and **Formatter**. The Planner breaks a question into sub-questions. The Retriever pulls documents from a fixed local corpus (no live web search). The Synthesizer drafts an answer with citations. The Formatter emits the final structured answer.

We built the graph in LangGraph and ran all LLM stages with Claude Sonnet (c`laude-sonnet-4-6`) at temperature 0, except where a pattern deliberately switches models (P3's cheap tier). Each stage writes typed fields into a shared state object, so a fallback can see what failed and where without guessing.

End-to-end success is a correct final answer against a labeled gold string (scoring details in Section 4). The design goal was a small production-like workflow where stage failures are easy to locate and recover from.

3.2 Conditions

We ran six conditions on the same questions and corpus. B0 and B1 are baselines. P1-P4 are the fallback patterns. Here we summarize what each condition does.

B0 -- No fallback. The pipeline runs once. Any hard stage error, or a wrong final answer, counts as failure. No retries. This is the reference point for cost and latency.

B1 -- Blind retry. On stage failure (or a failed final check), we re-run the same stage or the full graph up to a fixed number of times, with the same prompts and tools. No new evidence, no rollback to a trusted checkpoint, no second agent. We include it as the obvious "just retry" baseline that targeted patterns should improve on, on both reliability and cost.

P1 -- Tool-grounded retry-with-verification. After a stage produces a candidate output, we run cheap external checks: empty or low-relevance retrieval, citation IDs missing from the retrieved set, schema/JSON parse failures. If a check fails, we re-run only that stage and append the check result as feedback (CRITIC-style verify-then-correct). Retries are capped. This is not "please fix your answer" with no new signal.

P2 -- Deterministic checkpointing. After a stage passes verification, we save a checkpoint. On hard failure or verification failure at stage i , we discard untrusted state from i onward, restore the last trusted checkpoint, and re-execute forward from there (optionally with a one-shot tool hint). We do not keep polluted intermediate memory.

P3 -- Graceful degradation. After repeated recoveries fail at a stage, or a per-question token budget is exceeded, we switch to a simpler path: skip multi-hop planning, cascade remaining stages to a cheaper model (Haiku), and/or return a partial answer marked low-confidence. The task is still scored as pass/fail on the gold answer. When the Haiku path runs, those stages are priced at Haiku rates in the logs; the P3 cost and latency numbers we report already include that mix (they are not Sonnet-only totals with Haiku ignored). In the pooled runs, Haiku was used on a small minority of questions--most P3 runs never left the Sonnet path--so the average cost stays close to P1/P2.

P4 -- Cross-agent verification gate. Before the Synthesizer draft goes to the Formatter, a second Verifier agent (distinct system prompt, same Sonnet model) sees the question, retrieved evidence, and draft. It must ACCEPT, REJECT_WITH_CRITIQUE, or PROPOSE_CORRECTION. On reject, the Synthesizer re-runs once with the critique, still bound to the retrieved evidence. Reject-and-retry is capped at one round *per question*, so a single question cannot loop. That second synthesizer pass does fire on some questions; it just cannot repeat on the same item.

We evaluate each pattern as its own condition, not as stacked compositions. Composition (e.g. P2+P1, then P3) is left for later work once single-pattern effects are clear.

3.3 Why external / tool-grounded feedback (P1-P3)

P1-P3 deliberately avoid intrinsic self-correction as the main recovery signal. Huang et al. (2024) show that when models are asked to fix their own reasoning without external feedback, they often do not improve and can get worse. Kamoi et al. (2024) make the same point in survey form: reliable correction needs external signals, task-specific checks, or other grounding--not prompted self-evaluation alone.

So when P1 retries a stage, the feedback is a checkable fact (empty retrieval, bad citation ID, broken JSON schema), not "try again." P2 only writes checkpoints after an external pass. P3 degrades on budget or repeated external failures, not on the model's own confidence score. P4 is different: it uses a second agent as a gate. That is still external to the Synthesizer's first draft, but it is an LLM judge rather than a tool check, and we treat it as a separate (expensive) pattern family.

4. Experimental Setup

4.1 Dataset

We use 40 multi-hop questions from HotpotQA (distractor setting, validation split), loaded via Hugging Face datasets. Selection filters, recorded in the subset metadata:

- Short gold answers (at most 60 characters / 6 tokens)
- Yes/no answers excluded
- At least two supporting titles
- Bridge-type questions preferred; then sort and take the first 40

The retrieval corpus has 100 documents: 80 supporting documents tied to the selected questions, plus 20 distractor documents (sorted titles, stride subsample). The Retriever only searches this fixed corpus, so runs are reproducible and do not depend on a live search API.

4.2 Scoring

A question passes if the model's final answer matches the gold string under a normalized token-coverage rule: exact match, substring containment either way, or every gold token appearing in the prediction. That is lenient toward verbose answers and strict about covering the gold content.

This scorer is a limitation. It can mark a factually reasonable answer as fail when the wording diverges from Hotpot's short gold span, and it does not check citation quality on its own. We return to that in Section 7; the same scorer is used for every condition so relative comparisons stay fair.

4.3 Trial design and statistics

All LLM calls use temperature 0. We still ran multiple full trials because scoring flukes and rare parse paths can move a few questions.

- B0, B1, P1, P2, P3: three independent trials of the same 40 questions (pooled $n = 120$ question-outcomes per condition)
- P4: two trials (pooled $n = 80$)

The third trial for B0/B1/P1/P2/P3 was targeted after the two-trial screen showed edge-band significance for some B0 comparisons; P4 was not re-run.

For pass rate, we pool question outcomes across trials and compare conditions with a two-sided two-proportion z-test and Fisher's exact test ($\alpha = 0.05$). We do not report a run-level standard deviation from two or three trials. We did not apply multiple-comparison correction; pairwise tests are an exploratory screen and should be read that way.

For latency and cost we report per-trial values and simple averages. With only two or three trial totals, we do not attach confidence intervals or claim significant cost differences from the trial sample alone.

4.4 Metrics

We track three primary metrics:

1. **Pass rate** -- fraction of questions scored correct (pooled across trials)
2. **Latency** -- end-to-end wall time per question; we report p50 and p95 (pooled question-level distributions, and per-trial p50/p95 for inspection)
3. **Cost** -- estimated USD from logged token counts and published Claude pricing; we report total cost per trial and average cost per question

Secondary logging for P4 separates Verifier token/cost from the Synthesizer so verification overhead can be stated on its own line. That does not change the primary cost metric above.

5. Results

5.1 Headline

The main result is simple. The three targeted fallback patterns--tool-grounded retry (P1), deterministic checkpointing (P2), and graceful degradation (P3)-- raised pass rate over the no-fallback baseline (B0) by a clear margin, at a modest cost increase. Blind retry (B1) and cross-agent verification (P4) did not beat B0 on pass rate at $\alpha = 0.05$, even though both cost more and both stretch the latency tail.

Pass rates below are pooled question outcomes ($n = 120$ for B0/B1/P1/P2/P3; $n = 80$ for P4). Cost and latency are the pooled figures from the same runs. Pairwise tests are two-sided two-proportion z-tests and Fisher's exact tests, as in Section 4.

Table 1. Pooled results across trials (pass rate, latency, cost).

Cond	n	Pass%	p50 (ms)	p95 (ms)	\$/q
B0	120	60.0%	6512	10397	0.0060
B1	120	64.2%	7139	34506	0.0108
P1	120	75.8%	7093	10398	0.0075
P2	120	75.8%	6926	9591	0.0075
P3	120	78.3%	7236	10369	0.0075
P4	80	70.0%	10427	30708	0.0164

Table 2. Selected pairwise pass-rate tests (two-sided; $\alpha = 0.05$; no multiple-comparison correction).

Comparison	dpp	z	p(z)	p(Fisher)	sig
P1 vs B0	+15.8	2.627	0.0086	0.0125	**
P2 vs B0	+15.8	2.627	0.0086	0.0125	**
P3 vs B0	+18.3	3.075	0.0021	0.0032	**
P4 vs B0	+10.0	1.443	0.1489	0.1766	ns
B1 vs B0	+4.2	0.665	0.5059	0.5947	ns
P1 vs B1	+11.7	1.972	0.0486	0.0667	*
P3 vs B1	+14.2	2.425	0.0153	0.0221	*
P1 vs P2	+0.0	0.000	1.0000	1.0000	ns
P1 vs P3	-2.5	-0.461	0.6450	0.7589	ns
P2 vs P3	-2.5	-0.461	0.6450	0.7589	ns
P3 vs P4	+8.3	1.333	0.1824	0.1876	ns

5.2 Pass rate

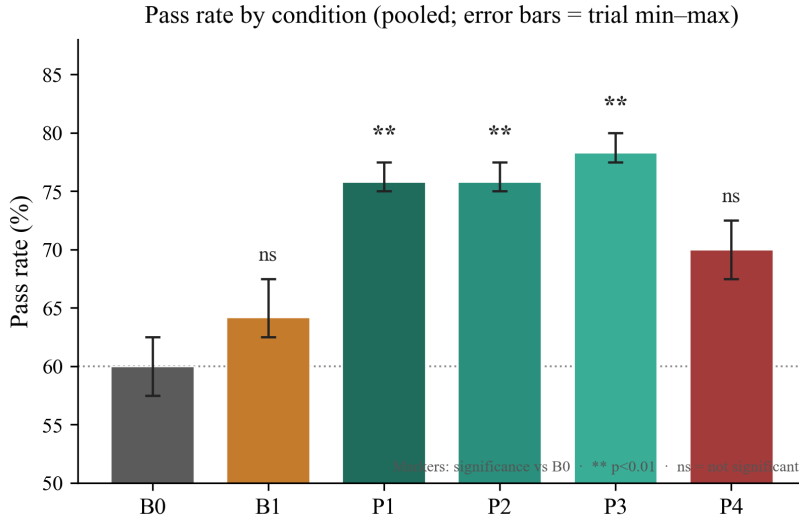


Figure 1: Pass rate by condition. Error bars span trial min-max; markers indicate significance vs B0.

Figure 1 shows pass rate by condition, with error bars spanning the trial min-max and significance markers vs B0.

B0 sits at 60.0% (72/120). P1 and P2 both land at 75.8% (91/120)--a +15.8 percentage-point gain over B0. Both comparisons are significant ($z = 2.627$, $p(z) = 0.0086$; Fisher $p = 0.0125$). P3 is a bit higher at 78.3% (94/120), +18.3 pp over B0, and the strongest of the three against the baseline ($z = 3.075$, $p(z) = 0.0021$; Fisher $p = 0.0032$).

B1 reaches 64.2% (77/120). That is only +4.2 pp over B0, and it is not significant ($p(z) = 0.5059$; Fisher $p = 0.5947$). P4 reaches 70.0% (56/80), +10.0 pp over B0, also not significant ($p(z) = 0.1489$; Fisher $p = 0.1766$). P4's sample is smaller (two trials), but the point estimate still does not clear the bar.

So on pass rate alone: P1, P2, and P3 beat doing nothing. Blind retry and the verifier gate do not, at least not at a level we can defend from these data.

5.3 Cost vs accuracy

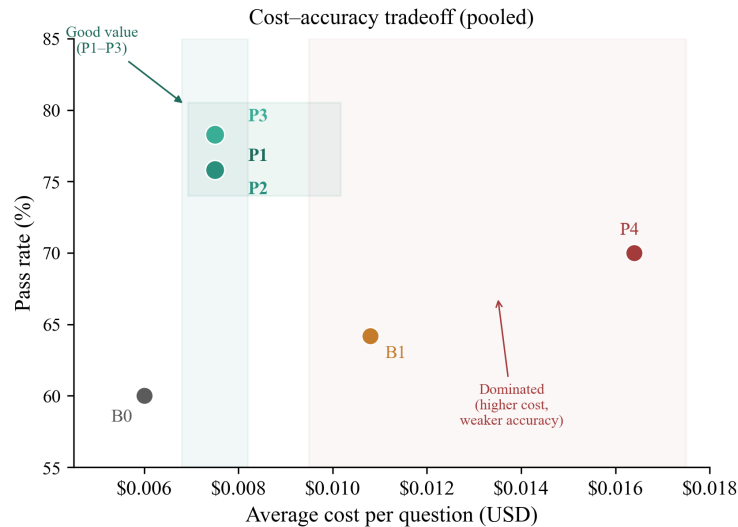


Figure 2: Cost vs accuracy (Pareto view). Average cost per question against pooled pass rate.

Figure 2 plots average cost per question against pass rate.

B0 is the cheap, lower-accuracy corner at \$0.0060/q and 60.0%. P1, P2, and P3 sit together near \$0.0075/q with pass rates of 75.8%, 75.8%, and 78.3%. That is the good-value cluster: roughly a quarter more spend than B0 for a fifteen-to-eighteen point pass-rate lift.

B1 costs \$0.0108/q for 64.2% pass rate--almost twice B0's cost for a gain that is not significant. P4 is the expensive outlier at \$0.0164/q and 70.0% pass rate. Both sit in the dominated region of Figure 2: higher cost than P1-P3, with worse or no better accuracy than that cluster. If the goal is reliability per dollar, B1 and P4 lose to the cheaper targeted patterns.

(These cost figures are descriptive. With only two or three trial totals per condition, we do not claim significant cost differences from a formal test on the trial sample.)

5.4 Latency

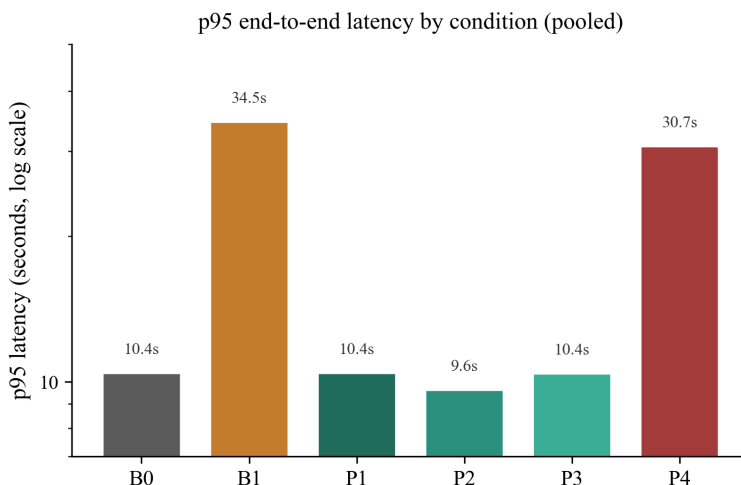


Figure 3: Pooled p95 end-to-end latency by condition (log scale).

Figure 3 shows pooled p95 end-to-end latency on a log scale.

B0, P1, P2, and P3 stay near ten seconds at p95: 10397 ms, 10398 ms, 9591 ms, and 10369 ms. B1 and P4 do not. B1's p95 is 34506 ms (~34.5 s). P4's is 30708 ms (~30.7 s). That is roughly three times the rest of the field.

The reasons match how each condition recovers. B1 re-runs the same stage or graph on failure, up to its retry cap, with no new evidence and no early targeted exit. Failed questions stack sequential retries, which fattens the tail even when the median stays closer to the pack (B1 p50 is 7139 ms). P4 puts a full Verifier agent call on the critical path before the Formatter, and on reject it may run a second Synthesizer pass (capped at one round per question). That extra agent work shows up as both higher average cost and a long p95, whether or not the gate eventually helps the answer.

5.5 P1, P2, and P3 are interchangeable on pass rate

P1, P2, and P3 are statistically indistinguishable from each other on pooled pass rate. P1 vs P2: both 75.8%, Fisher $p = 1.0000$. P1 vs P3: 75.8% vs 78.3%, Fisher $p = 0.7589$. P2 vs P3: same gap, Fisher $p = 0.7589$.

We are not claiming one of these three is best. The finding is that three different targeted designs--tool checks with stage retry, deterministic checkpointing, and degrade-on-exhaustion--land in the same place: clear gains over B0, similar cost, similar latency tails. Choosing among them is more about engineering fit than about a winner on this benchmark.

5.6 P4: expensive verification without a significant win

P4 is the counter-intuitive result. It is the most elaborate pattern in the set: a second LLM gates the Synthesizer draft before formatting, with accept / reject-with-critique / propose-correction, plus a capped re-synth on reject. It is also the most expensive condition (\$0.0164 per question) and one of the two worst latency tails (~30.7 s p95).

It did not significantly outperform B0 on pass rate (70.0% vs 60.0%; $p(z) = 0.1489$, Fisher $p = 0.1766$). Against P1-P3 it is numerically lower and not significantly different from them either (e.g. P3 vs P4: Fisher $p = 0.1876$), while costing more than twice as much per question as that cluster.

So in this setup, paying for a full cross-agent verification gate did not buy a defensible reliability gain over doing nothing--and it lost on cost and latency to simpler tool-grounded and checkpoint-style fallbacks. That is a real finding for the paper, not a failed run to soft-pedal.

6. Discussion

If you are building a multi-agent pipeline, the practical takeaway is narrow and useful. Do not default to blind retry. It did not significantly beat no fallback on pass rate, and it blew out p95 latency because failed questions stack sequential re-runs with no new evidence. Do not assume a cross-agent verifier is automatically "more reliable" because it looks sophisticated. In our runs it was the most expensive condition and one of the two worst latency tails, without a significant pass-rate win over doing nothing.

What worked was simpler: tool- or schema-grounded stage checks with a capped retry (P1), rollback to a last trusted checkpoint (P2), or degrade when retries and budget are exhausted (P3). Those paths raised pass rate over B0 at about \$0.0075 per question versus \$0.0060 for B0--more spend, but nowhere near B1 or P4--and kept p95 latency near ten seconds instead of thirty.

That P1, P2, and P3 are statistically indistinguishable matters for how you read the paper. This is not an accuracy horse race among three winners. It is evidence that several targeted designs converge on similar reliability gains when feedback is external and recovery is scoped to the failing stage or a trusted prefix. The engineering choice is which mechanism fits your graph--stage verify-and-retry, checkpoint restore, or degrade-on-exhaustion-- not which one "won" HotpotQA by a point or two.

P4's underperformance is easier to understand once you take LLM-as-judge limits seriously. Section 2 already notes that judge models carry biases and soft accepts. We hit a concrete version of that in development: the first verifier prompt rubber-stamped almost every draft (ACCEPT on every question that reached the gate), so the "gate" was not gating. After we tightened the checklist, added deterministic citation checks, and fail-closed on parse errors, the verifier did reject and occasionally re-ran the Synthesizer--but pooled pass rate still did not clear significance against B0, while cost and p95 stayed high. A second LLM on the critical path is not free, and it is only as useful as its willingness to say no for the right reasons. That is a design lesson, not just a non-significant p-value.

More broadly, multi-agent reliability looks less like "add another agent" and more like orchestration hygiene: external checks at stage boundaries, immutable checkpoints after trusted passes, and a planned degrade path when recovery fails. Patterns from CRITIC-style tool critique and AgentDebug-style targeted re-roll beat no-fallback here, and look better than blind retry on cost and latency--though the pass-rate edge over blind retry itself sits right at the edge of significance, not a clean win. Patterns that multiply full agent calls without cheap external signals did not pay for themselves on this testbed. Builders should budget verification the way they budget any other stage--against measured failure, latency, and cost--not against how careful the architecture diagram looks.

7. Limitations

These claims have sharp edges.

Narrow task and small set. We used 40 HotpotQA multi-hop questions and one pipeline shape (plan -> retrieve -> synthesize -> format). Coding agents, support triage, or tool-heavy workflows may fail differently. We do not claim the same ranking holds there.

Scorer. Pass/fail uses normalized token-coverage matching against short gold answers. It can fail a reasonable paraphrase and does not score citation quality on its own. Relative comparisons across conditions stay fair because every condition uses the same scorer; absolute pass rates should be read with that limit in mind.

A manual spot-check of B0's six score-fail questions (the same fails recur across trials) found no clear model hallucinations. Most were name-form mismatches--for example "Kareena Kapoor" versus gold "Kareena Kapoor Khan"--or cases where the retrieved evidence backed the model's answer over Hotpot's stated gold (two cases: a Beethoven work mislabeled in the gold set, and a diplomatic-mission question where the gold tribe name conflicted with the supporting document). So reported pass rates are, if anything, a slight underestimate of true answer quality, not an overestimate--and the same scorer and gold set are used identically across all six conditions.

Single patterns only. We did not test compositions (e.g. P1 inside P2, then P3 as last resort). Stacked policies might do better--or cost more for little gain. That is left open.

Modest trial counts. B0/B1/P1/P2/P3 have three trials (pooled $n = 120$); P4 has two ($n = 80$). That is enough for the comparisons we report at temperature 0, and weaker than large multi-seed ML evaluations. A few pairwise edges (notably P1/P2 vs B1) still sit near the decision boundary.

Single base model. All primary stages use Claude Sonnet (claude-sonnet-4-6), with Haiku only on P3's rare cheap path. Other models may reject, parse, or judge differently. Pattern rankings could shift.

No multiple-comparison correction. Pairwise pass-rate tests are an exploratory screen at $\alpha = 0.05$ without family-wise adjustment. Significant stars should be read that way, not as a fully corrected tournament among six conditions.

8. References

- Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. Transactions on Machine Learning Research (TMLR), 2024. arXiv:2305.05176.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235, 2024. arXiv:2305.14325.
- Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., and Chen, W. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In International Conference on Learning Representations (ICLR), 2024. arXiv:2305.11738.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large Language Models Cannot Self-Correct Reasoning Yet. In International Conference on Learning Representations (ICLR), 2024. arXiv:2310.01798.
- Kamoi, R., Zhang, Y., Zhang, N., Han, J., and Zhang, R. When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs. Transactions of the Association for Computational Linguistics (TACL), 2024. <https://aclanthology.org/2024.tacl-1.78/>.
- Roy, D., Zhang, X., Bhawe, R., Bansal, C., Las-Casas, P., Fonseca, R., and Rajmohan, S. Exploring LLM-based Agents for Root Cause Analysis. In Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track, 2024. <https://www.microsoft.com/en-us/research/publication/exploring-llm-based-agents-for-root-cause-analysis/>.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2303.11366.
- Song, P., Han, P., and Goodman, N. Large Language Model Reasoning Failures. Transactions on Machine Learning Research (TMLR), 2026. arXiv:2602.06176.
- Tan, G., He, Z., Li, M., Huang, H., Wang, Y., Chen, P., Casale, G., and Zhang, C. LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data. ACM Transactions on Software Engineering and Methodology (TOSEM), 2026. DOI:10.1145/3806200.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J.-R. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024. arXiv:2308.11432.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In International Conference on Learning Representations (ICLR), 2023. arXiv:2203.11171.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023. arXiv:2306.05685.
- Zhu, K., Liu, Z., Li, B., Tian, M., Yang, Y., Zhang, J., Han, P., Xie, Q., Cui, F., Zhang, W., Ma, X., Yu, X., Ramesh, G., Wu, J., Liu, Z., Lu, P., Zou, J., and You, J. Where LLM Agents Fail and How They Can Learn From Failures. arXiv preprint arXiv:2509.25370, 2025 (AgentDebug).